CI/CD expressions

CI/CD expressions enable dynamic configuration in your CI/CD pipelines by referencing variables and inputs in specialized contexts. GitLab evaluates expressions in the pipeline configuration before the pipeline is created.

Configuration expressions

Configuration expressions use the `$[[ ]]` syntax and are evaluated at pipeline creation time (compile-time). They enable dynamic configuration based on different contexts.

All configuration expressions share these characteristics:

- **Compile-time evaluation**: Values are resolved when the pipeline configuration is created, not during job execution. A large number of expressions can increase pipeline creation time, but does not affect job execution time.
- **Static resolution**: Cannot perform dynamic logic or access runtime job state.

Configuration expressions support different contexts for accessing values:

Context	Syntax	Availability	Purpose
Inputs context	`$[[ inputs.INPUT_NAME ]]`	GitLab 17.0	Reference CI/CD inputs in reusable configurations.
Matrix context	`$[[ matrix.IDENTIFIER ]]`	GitLab 18.6 (Beta)	Reference `parallel:matrix` identifiers in job dependencies.
Component context	`$[[ component.FIELD_NAME ]]`	GitLab 18.6 (Beta)	Reference component metadata in component templates.

Inputs context

History >

Use the `inputs.` context to reference CI/CD inputs in reusable configurations using `$[[ inputs.INPUT_NAME ]]` syntax.

For example:

```yaml
spec:
  inputs:
    environment:
      default: production
    job-stage:
      default: test
---
scan-website:
  stage: $[[ inputs.job-stage ]]
  script: ./scan-website $[[ inputs.environment ]]
```

`input.` expressions have the following characteristics:

- Type validation: Supports `string`, `number`, `boolean`, and `array` types with validation. Input validation prevents pipeline creation with invalid values.

Matrix context

History >

Use the `matrix.` context to reference `parallel:matrix` values by using a `$[[ matrix.IDENTIFIER ]]` syntax. Use it in job dependencies to enable dynamic 1:1 mappings between `parallel:matrix` jobs.

For example:

```yaml
.os-arch-matrix:
  parallel:
    matrix:
      - OS: [ubuntu, alpine]
        ARCH: [amd64, arm64]

build:
  script: echo "Testing $OS on $ARCH"
  parallel: !reference [.os-arch-matrix, parallel]

test:
  script: echo "Testing $OS on $ARCH"
  parallel: !reference [.os-arch-matrix, parallel]
  needs:
    - job: build
      parallel:
        matrix:
          - OS: ['$[[ matrix.OS ]]']
            ARCH: ['$[[ matrix.ARCH ]]']
```

`matrix.` expressions have the following characteristics:

- Scoped to job-level `parallel:matrix`: Only values from the current job can be referenced.
- Automatic mapping: Creates 1:1 dependencies between matrix jobs across stages

Component context

History >

Use the `component.` context to reference CI/CD component metadata in component templates using `$[[ component.FIELD_NAME ]]` syntax.

Component context provides metadata about the component itself, such as its name, version, and the commit SHA. This allows component templates to reference their own metadata dynamically.

To use component context, declare which fields are needed in the `spec:component` header, then reference them in the component template.

For example:

```yaml
```

```
    stage:
      default: build
---

build-job:
  stage: $[[ inputs.stage ]]
  image: registry.example.com/$[[ component.name ]]:$[[ component.version ]]
  script:
    - echo "Building with component version $[[ component.version ]]"
```

Related topics

- Moa expression language
- CI/CD inputs
- CI/CD components
- Matrix expressions

BY-SA